

# Registers

Storing many bits · shift registers · SISO, SIPO, PISO, PIPO · bidirectional · universal shift register

## QUICK TRICK

**Read the four letters like this:** the first two letters tell how data goes **IN**, the last two how it comes **OUT**. **Serial** = one bit at a time (**n** clock pulses). **Parallel** = all bits at once (**1** clock pulse).

## 14.1 What is a register?

### ● LEVEL 1 · BEGINNER

**What is it?** A flip-flop stores **one bit**. A **register** is a group of flip-flops that stores **many bits** together. An 8-bit register has 8 flip-flops and can store 10110101.

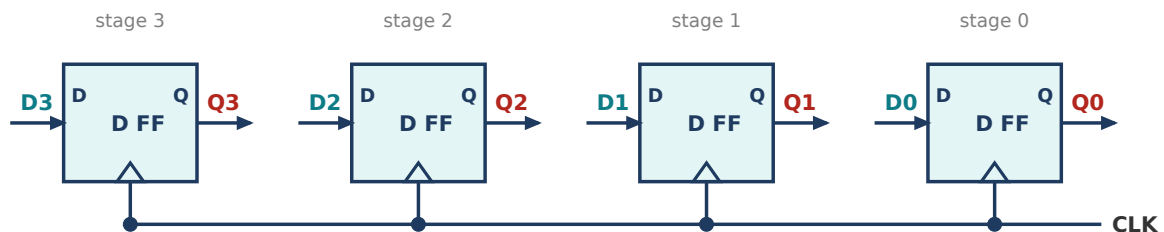
**Why do we need it?** A computer works on numbers like 8, 16 or 32 bits wide. Registers hold these numbers inside the processor.

**Analogy.** A row of lockers. Each locker holds one bit. A register is the whole row.

## EASY MEMORY

$n$  flip-flops =  $n$ -bit register. It can hold  $2^n$  different values. All flip-flops share the **same clock**.

**The simplest register** (called PIPO, parallel in – parallel out) loads all bits at the same clock pulse:



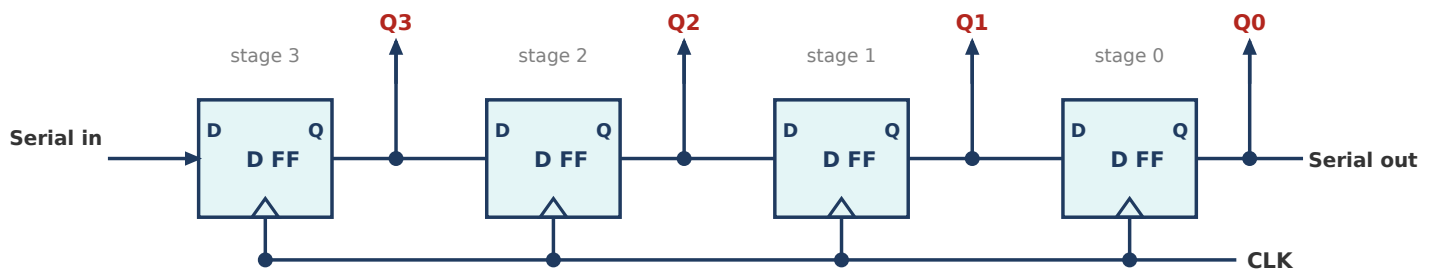
## 14.2 Shift register

### ● LEVEL 1 · BEGINNER

**What is it?** A register in which the stored bits **move one place** at every clock pulse. The output of each flip-flop is connected to the input of the next one.

**Why?** To move data in or out one bit at a time (using only one wire), to delay a signal, and to multiply or divide by 2.

**Analogy.** A queue of people. When the front person leaves, everyone steps forward and a new person joins at the back.



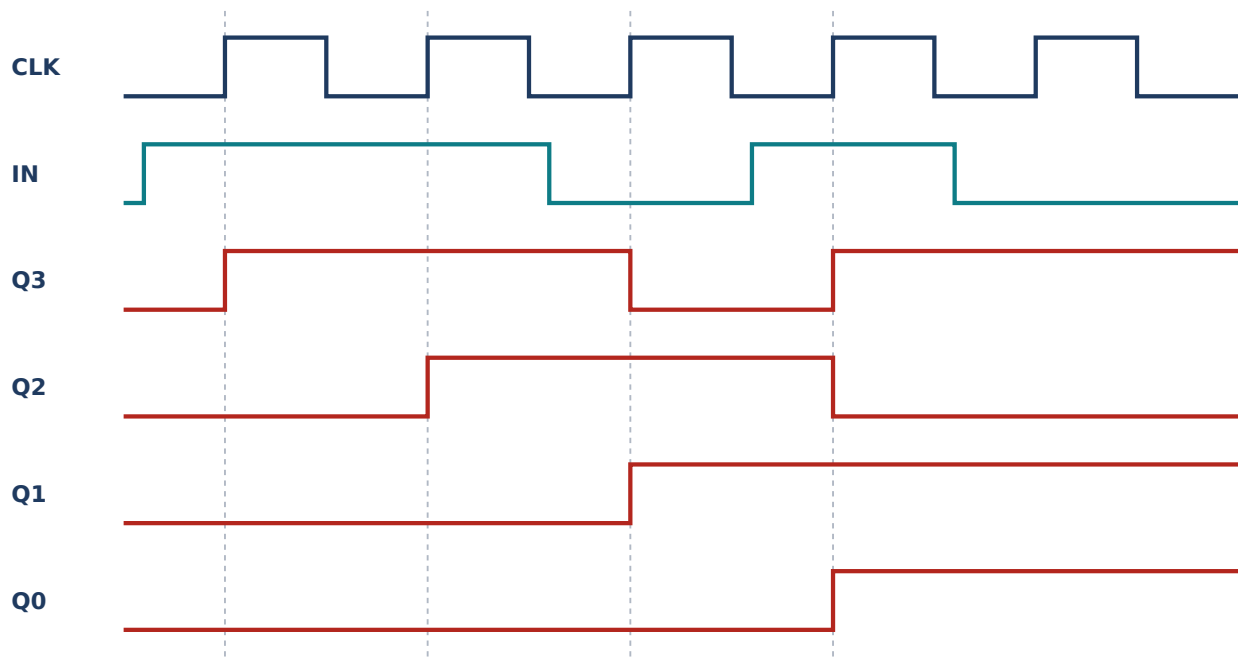
Here data enters at the left and moves to the right. The **first bit that enters travels through all the flip-flops** and reaches the last one.

### Example: loading 1011 serially into a 4-bit register

The register starts at 0000. We send the bits **1, 1, 0, 1** (the bit that must end in Q0 goes in **first**).

| Clock | Serial in | Q3 | Q2 | Q1 | Q0 |
|-------|-----------|----|----|----|----|
| 0     | —         | 0  | 0  | 0  | 0  |
| 1     | 1         | 1  | 0  | 0  | 0  |
| 2     | 1         | 1  | 1  | 0  | 0  |
| 3     | 0         | 0  | 1  | 1  | 0  |
| 4     | 1         | 1  | 0  | 1  | 1  |

After 4 clock pulses the register holds **Q3 Q2 Q1 Q0 = 1011**. Here is the timing diagram. At every rising edge (dashed line) each flip-flop copies the bit on its left, using the value it had **just before** the edge.



#### QUICK TRICK

**Rule for a right-shift register:** the first bit entered ends up in the **last** flip-flop (Q0) after n clock pulses. So to get 1011 you send the bits in the order of Q0 first: 1, 1, 0, 1.

## 14.3 The four types of shift register

### ● LEVEL 2 • EXAM

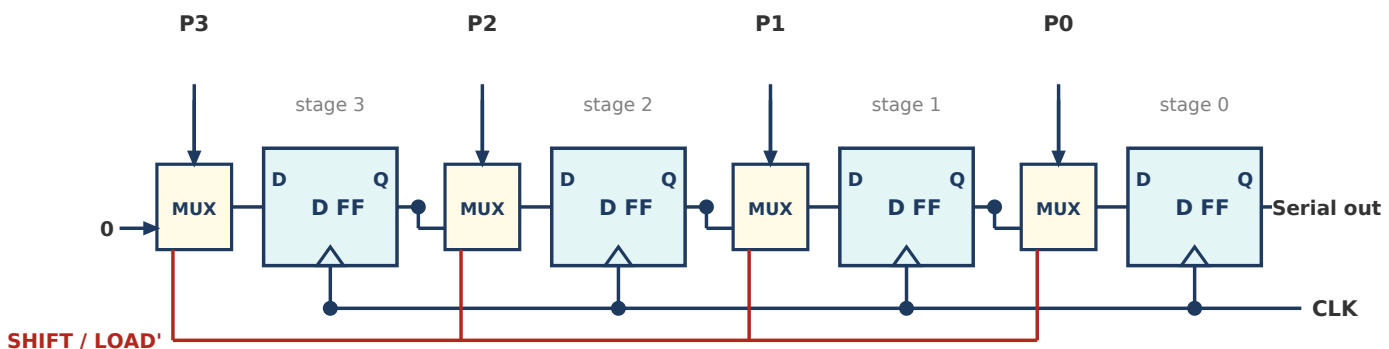
| Type        | Data goes IN | Data comes OUT | Clock pulses to load | Clock pulses to read | Used for                     |
|-------------|--------------|----------------|----------------------|----------------------|------------------------------|
| <b>SISO</b> | Serial       | Serial         | n                    | n                    | Delay line, simple storage   |
| <b>SIPO</b> | Serial       | Parallel       | n                    | 1 (all bits at once) | Serial → parallel conversion |
| <b>PISO</b> | Parallel     | Serial         | 1                    | n                    | Parallel → serial conversion |
| <b>PIPO</b> | Parallel     | Parallel       | 1                    | 1                    | Ordinary storage register    |

#### QUICK TRICK

**Easy memory:** S-I-P-O = Serial In, Parallel Out. Count the clock pulses by looking at the word **serial**: each serial side needs **n** pulses, each parallel side needs **1**.

### PISO: parallel in, serial out

Each flip-flop gets a small **2:1 MUX** in front. A control line **SHIFT/LOAD'** chooses: **0 = load** the parallel data P3..P0 (all at once, 1 clock pulse); **1 = shift** (the bit from the left neighbour). After loading, the bits come out one by one at the serial output.



#### GATE POINT

**Serial ↔ parallel conversion:** SIPO converts a stream of bits into a parallel word (needs n clocks). PISO does the opposite. Computers use these to talk over a single wire.

## 14.4 Bidirectional shift register

### ● LEVEL 2 • EXAM

**What is it?** A shift register that can shift **right or left**. A control input M chooses the direction. Each flip-flop gets a **2:1 MUX** in front: it can take the bit from its **left** neighbour (shift right) or from its **right** neighbour (shift left).

| M | Direction   | What happens (Q3 Q2 Q1 Q0)                 |
|---|-------------|--------------------------------------------|
| 0 | Shift right | Q3 → Q2 → Q1 → Q0. A new bit enters at Q3. |
| 1 | Shift left  | Q0 → Q1 → Q2 → Q3. A new bit enters at Q0. |

### QUICK TRICK

**Shift = multiply or divide by 2.** Shift **left** by 1 =  $\times 2$ . Shift **right** by 1 =  $\div 2$  (the dropped bit is the remainder).

0011 (3) shift left  $\rightarrow$  0110 (6). 1100 (12) shift right  $\rightarrow$  0110 (6). Shifting by  $k$  places multiplies or divides by  $2^k$ .

| Kind of shift               | What goes into the empty place                                     | Example (8 bits): 10110011 shifted by 2 |
|-----------------------------|--------------------------------------------------------------------|-----------------------------------------|
| Logical shift right         | 0s                                                                 | <b>00101100</b>                         |
| Arithmetic shift right      | A copy of the sign bit (keeps the sign of a 2's complement number) | <b>11101100</b>                         |
| Rotate (circular) left by 1 | The bit that fell off the other end                                | 10010110 $\rightarrow$ <b>00101101</b>  |

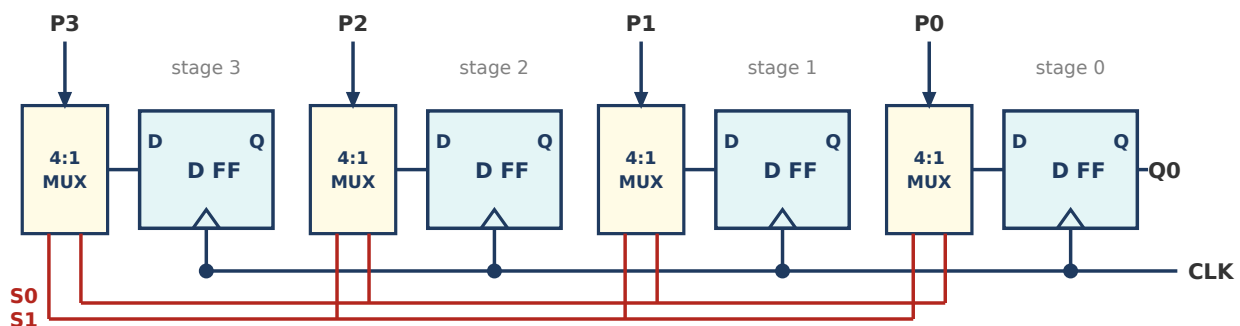
### EXAM TRAP

**Arithmetic shift right** copies the sign bit, but a **logical shift** puts 0s. They give different answers for negative numbers. A **rotate** loses no bit: it joins the output back to the input.

## 14.5 Universal shift register

### ● LEVEL 3 · GATE

**What is it?** One register that can do **everything**: hold, shift right, shift left and parallel load. (The IC 74194 is a 4-bit universal shift register.) Each flip-flop gets a **4:1 MUX**; two select lines S1 S0 choose the mode.



| S1 | S0 | Mode                 | What each flip-flop takes as input                                 |
|----|----|----------------------|--------------------------------------------------------------------|
| 0  | 0  | <b>Hold</b>          | Its own output (no change)                                         |
| 0  | 1  | <b>Shift right</b>   | The output of its left neighbour (serial-right input enters at Q3) |
| 1  | 0  | <b>Shift left</b>    | The output of its right neighbour (serial-left input enters at Q0) |
| 1  | 1  | <b>Parallel load</b> | The parallel input P                                               |

### QUICK TRICK

**Mode code: Hold, Right, Left, Load = 0, 1, 2, 3.** Read S1 S0 as a binary number. 00 = 0 = Hold. 01 = 1 = shift Right. 10 = 2 = shift Left. 11 = 3 = Load. An  $n$ -bit universal register needs  $n$  **4:1 MUXes**.

**Example run** (Q3 Q2 Q1 Q0):

| Step  | S1 S0 | Mode        | Serial/parallel inputs | Q3 Q2 Q1 Q0 |
|-------|-------|-------------|------------------------|-------------|
| start | —     | —           | —                      | 0000        |
| 1     | 1 1   | Load        | P3..P0 = 1010          | 1010        |
| 2     | 0 1   | Shift right | serial in (right) = 1  | 1101        |
| 3     | 1 0   | Shift left  | serial in (left) = 0   | 1010        |
| 4     | 0 0   | Hold        | —                      | 1010        |

## 14.6 A shift register can also count (preview of Chapter 15)

### ● LEVEL 3 · GATE

If the **last output is fed back to the first input**, the bits go round and round. This makes counters:

| Counter                | Feedback           | Sequence for 4 bits                                            | States |
|------------------------|--------------------|----------------------------------------------------------------|--------|
| <b>Ring counter</b>    | Q0 → serial input  | 1000 → 0100 → 0010 → 0001 → repeat (start 1000)                | n      |
| <b>Johnson counter</b> | Q0' → serial input | 0000 → 1000 → 1100 → 1110 → 1111 → 0111 → 0011 → 0001 → repeat | 2n     |

#### GATE POINT

n flip-flops: **ring counter = n states**, **Johnson counter = 2n states** (the inverted feedback doubles the count). (Chapter 15 explains them in detail.)

## Quick-Solve Tricks — Summary

| Topic                          | Trick                                                     |
|--------------------------------|-----------------------------------------------------------|
| Bits stored                    | n flip-flops = n-bit register.                            |
| Serial vs parallel             | Serial = n clock pulses. Parallel = 1 clock pulse.        |
| First bit in                   | Ends in the last flip-flop after n clock pulses.          |
| Shift = $\times 2$ or $\div 2$ | Left $\times 2$ , right $\div 2$ (k places = $2^k$ ).     |
| Universal register             | S1 S0 = 00 Hold, 01 Right, 10 Left, 11 Load. n 4:1 MUXes. |
| Ring / Johnson                 | Ring = n states, Johnson = 2n states.                     |

## GATE QUICK CHECK — Chapter 14

n-bit register = n flip-flops                      Serial = n clocks, Parallel = 1 clock  
 Shift left =  $\times 2$ , shift right =  $\div 2$   
 Delay of an n-bit shift register = n clock periods  
 Universal register S1 S0 : 00 hold, 01 right, 10 left, 11 load  
 Ring counter n states, Johnson counter 2n states

**Traps:** order of the bits when loading serially · PISO needs a MUX and a load control · arithmetic vs logical shift · Johnson counter has 2n states (not n).

## Practice questions

### ● LEVEL 1 • BEGINNER

**Q1.** How many flip-flops are needed for an 8-bit register?

**Q2.** What does PISO stand for?

**Q3.** What is the effect of shifting 0011 left by one place?

### ● LEVEL 2 • EXAM

**Q4.** A 4-bit right-shift register starts at 0000. The serial input is 1, 0, 1, 1 (in this order), entering at Q3. Find Q3 Q2 Q1 Q0 after 4 clock pulses.

**Q5.** A 10 MHz clock drives an 8-bit SISO register. How long does it take to load 8 bits serially?

**Q6.** A 4-bit universal shift register holds 0110. S1 S0 = 10 (shift left) and the serial-left input is 1. Find the new value.

### ● LEVEL 3 • GATE

**Q7.** A 4-bit ring counter starts at 1000. Find the value after 3 clock pulses.

**Q8.** A 4-bit Johnson counter starts at 0000. Find the value after 3 clock pulses.

**Q9.** An 8-bit register holds 10110011. Find the result of (a) a logical shift right by 2 and (b) an arithmetic shift right by 2.

**Q10.** Rotate 10010110 left by one place.

**Q11.** How many 4:1 MUXes does a 4-bit universal shift register need?

**Q12.** A 4-bit PISO register (right shift, serial output at Q0) is loaded in parallel with Q3 Q2 Q1 Q0 = 1011. In what order do the bits come out?

## Answers

| Q  | Solution                                                                                                                                                                           |
|----|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1  | <b>8</b> flip-flops.                                                                                                                                                               |
| 2  | <b>Parallel In, Serial Out.</b>                                                                                                                                                    |
| 3  | Shift left = $\times 2$ : 0011 (3) → <b>0110</b> (6).                                                                                                                              |
| 4  | After clock 1: 1000. Clock 2 (input 0): 0100. Clock 3 (input 1): 1010. Clock 4 (input 1): <b>1101</b> .                                                                            |
| 5  | Clock period = $1 / 10 \text{ MHz} = 100 \text{ ns}$ . 8 pulses $\times 100 \text{ ns} =$ <b>800 ns</b> .                                                                          |
| 6  | Shift left: $Q3 \leftarrow Q2 = 1$ , $Q2 \leftarrow Q1 = 1$ , $Q1 \leftarrow Q0 = 0$ , $Q0 \leftarrow \text{serial-left} = 1$ . New value <b>1101</b> ( $= 6 \times 2 + 1 = 13$ ). |
| 7  | $1000 \rightarrow 0100 \rightarrow 0010 \rightarrow$ <b>0001</b> .                                                                                                                 |
| 8  | $0000 \rightarrow 1000 \rightarrow 1100 \rightarrow$ <b>1110</b> .                                                                                                                 |
| 9  | (a) Logical: <b>00101100</b> . (b) Arithmetic (sign bit 1 is copied): <b>11101100</b> .                                                                                            |
| 10 | The leftmost 1 goes to the right end: $1001 \ 0110 \rightarrow$ <b>0010 1101</b> .                                                                                                 |
| 11 | One 4:1 MUX per flip-flop → <b>4</b> .                                                                                                                                             |
| 12 | Q0 comes out first: <b>1, 1, 0, 1</b> ( $Q0 = 1$ , $Q1 = 1$ , $Q2 = 0$ , $Q3 = 1$ ). The first bit is available right after loading; the remaining 3 need 3 more clock pulses.     |

*Next: Chapter 15 — Counters (ripple/asynchronous, synchronous, up/down, mod-N, decade, ring, Johnson, frequency division, design).*